SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.RAJU	**Reg. No.:**	192411065
Section / Batch:	B. TECH(CSE)	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

1. Knapsack Comparison — DP vs Greedy

Given

- Weights = [10, 20, 30]
- Values = [60, 100, 120]
- Capacity = 50

Value/weight ratios:

Item	Weight	Value	Value/Weight
1	10	60	6
2	20	100	5
3	30	120	4

A. 0/1 Knapsack using Dynamic Programming

In 0/1 Knapsack, an item is either completely selected or completely rejected.

Possible combinations within capacity 50:

- Item 1 + Item 2 = weight 30, value 160
- Item 1 + Item 3 = weight 40, value 180

- Item 2 + Item 3 = weight 50, value 220
- Item 1 + Item 2 + Item 3 = weight 60 → exceeds capacity

Therefore:

Optimal solution = Items 2 and 3

- Total weight = 20 + 30 = 50
- Maximum value = 100 + 120 = 220

B. Fractional Knapsack using Greedy

Select items according to highest value/weight ratio.

1. Take Item 1 → weight 10, value 60
2. Take Item 2 → weight 20, value 100
3. Remaining capacity = 20
4. Item 3 weighs 30, so take 20/30 of Item 3

Value:

60 + 100 + (20/30 × 120) = 240

Therefore:

Optimal fractional value = 240

Comparison

Feature	0/1 Knapsack	Fractional Knapsack
Method	Dynamic Programming	Greedy
Items divisible?	No	Yes
Result	220	240
Time complexity	O(nW)	O(n log n)
Approach	Considers combinations	Uses value/weight ratio
Guarantee	Optimal for 0/1 problem	Optimal for fractional problem

Analysis

Greedy is optimal for Fractional Knapsack because an item can be divided. The highest value/weight ratio can always be selected first.

For 0/1 Knapsack, greedy does not always produce the optimal solution because items cannot be divided.

Strengths and Limitations

Dynamic Programming

- Gives an optimal 0/1 solution.
- Handles overlapping subproblems efficiently.
- Requires more memory.
- Can be expensive when capacity W is very large.

Greedy

- Simple and fast.
- Requires less memory.
- Excellent for Fractional Knapsack.
- Does not guarantee an optimal answer for general 0/1 Knapsack.

Insight

Use Greedy when the problem has the greedy-choice property, such as Fractional Knapsack. Use DP when decisions interact and local choices may prevent the global optimum.

2. Shortest Path — Dijkstra vs Bellman-Ford

Consider the following directed graph:

- A → B = 4
- A → C = 5
- B → C = -3
- C → D = 4
- B → D = 5

There is a negative-weight edge B → C = -3.

A. Dijkstra's Algorithm

Starting from A:

Initial distances:

- A = 0
- B = 4
- C = 5
- D = ∞

Dijkstra chooses B because it has the smallest temporary distance.

From B:

C = 4 + (-3) = 1

D = 4 + 5 = 9

Then C is selected:

D = 1 + 4 = 5

Final distances:

Vertex	Distance
A	0
B	4
C	1
D	5

For this particular graph, Dijkstra happens to obtain the correct distances.

However, this does not mean Dijkstra is valid for graphs containing negative edges.

Example showing Dijkstra's failure

Consider:

- A → B = 2
- A → C = 5
- C → B = -10

Actual shortest path:

A → C → B

Distance:

5 + (-10) = -5

But Dijkstra initially selects B with distance 2 and may finalize B before discovering the better path through C.

Thus, Dijkstra can produce an incorrect answer.

B. Bellman-Ford

Bellman-Ford repeatedly relaxes all edges.

For the example:

A = 0

After relaxation:

B = 4

C = min(5, 4-3) = 1

D = 1+4 = 5

Final result:

Vertex	Shortest Distance
A	0
B	4
C	1
D	5

Bellman-Ford also works when negative edges are present.

It can additionally detect negative-weight cycles.

Comparison

Feature	Dijkstra	Bellman-Ford
Negative edges	✗ No	✅ Yes
Negative cycle detection	✗ No	✅ Yes
Time complexity	O((V+E) log V) with heap	O(VE)
Speed	Faster	Slower
Main use	Non-negative weighted graphs	Graphs with negative edges
Approach	Greedy	Dynamic Programming/relaxation

Why does Dijkstra fail?

Dijkstra assumes that once the vertex with the smallest tentative distance is selected, its distance cannot be improved later.

A negative edge violates this assumption because a later path can reduce an already finalized distance.

Conclusion

- Use Dijkstra when all edge weights are non-negative and speed is important.
- Use Bellman-Ford when negative edges may occur or negative-cycle detection is required.

3**. MST Comparison — Prim vs Kruskal**

Consider the undirected weighted graph:

Edge	Weight
A-B	1
A-C	4
B-C	2
B-D	5
C-D	3
C-E	6
D-E	2

A. Prim's Algorithm

Start from vertex A.

Step 1

Available edges:

- A-B = 1
- A-C = 4

Select A-B = 1

Step 2

From {A,B}:

- A-C = 4
- B-C = 2
- B-D = 5

Select B-C = 2

Step 3

From {A,B,C}:

- C-D = 3
- B-D = 5
- C-E = 6

Select C-D = 3

Step 4

From {A,B,C,D}:

- D-E = 2
- C-E = 6

Select D-E = 2

Prim's MST

Edges:

A-B, B-C, C-D, D-E

Total weight:

1 + 2 + 3 + 2 = 8

B. Kruskal's Algorithm

Kruskal sorts all edges by weight:

1. A-B = 1
2. B-C = 2
3. D-E = 2
4. C-D = 3
5. A-C = 4
6. B-D = 5
7. C-E = 6

Select edges while avoiding cycles.

Step 1

A-B → select

Step 2

B-C → select

Step 3

D-E → select

Step 4

C-D → select

Now all 5 vertices are connected.

MST:

A-B, B-C, D-E, C-D

Total weight:

1 + 2 + 2 + 3 = 8

Comparison

Feature	Prim	Kruskal
Strategy	Vertex-based	Edge-based
Starting point	Required	Not required
Main operation	Select minimum edge connecting tree	Select globally smallest edge
Cycle handling	Naturally avoided	Union-Find
Complexity	O(E log V) with heap	O(E log E)
Good for	Dense graphs	Sparse graphs
Implementation	Priority Queue	Sorting + Disjoint Set

Dense vs Sparse Graphs

Prim

- Often preferred for dense graphs.
- Can efficiently grow the MST from a starting vertex.

Kruskal

- Very effective for sparse graphs.
- Sorting edges is straightforward.
- Union-Find makes cycle detection efficient.

Conclusion

Both algorithms produce an MST of total weight 8. The choice depends mainly on graph structure and implementation requirements.

4. Coin Change — Greedy vs Dynamic Programming

Given

Coins:

[1, 3, 4]

Amount:

6

A. Greedy Approach

Always select the largest possible coin.

Amount = 6

Step 1

Choose 4.

Remaining:

6 - 4 = 2

Step 2

Choose 1.

Remaining:

2 - 1 = 1

Step 3

Choose 1.

Remaining:

1 - 1 = 0

Therefore:

Greedy solution = 4 + 1 + 1

Number of coins = 3

B. Dynamic Programming

DP finds the minimum number of coins for every amount from 0 to 6.

Amount	Minimum coins
0	0
1	1
2	2
3	1
4	1
5	2
6	2

For amount 6:

3 + 3 = 6

Therefore:

DP solution = 3 + 3

Number of coins = 2

Comparison

Method	Solution	Number of Coins
Greedy	4 + 1 + 1	3
DP	3 + 3	2

Why does Greedy fail?

Greedy makes the locally best decision by choosing the largest coin, 4.

But this leaves an amount of 2, requiring two additional coins.

The globally optimal solution is:

3 + 3

Thus, choosing the largest coin first does not always lead to the minimum total number of coins.

Conclusion

Greedy works for some coin systems, such as common currency denominations, but it is not guaranteed to work for arbitrary denominations.

For arbitrary denominations, Dynamic Programming is safer because it explores the optimal combinations.

5. Activity Selection — Greedy vs DP

Given

Activity	Start	Finish
A1	1	2
A2	3	4
A3	0	6
A4	5	7
A5	8	9
A6	5	9

A. Greedy Approach

The activity-selection problem uses the greedy rule:

Always select the activity that finishes earliest.

Sort activities by finish time:

Activity	Start	Finish
A1	1	2
A2	3	4
A3	0	6
A4	5	7
A5	8	9
A6	5	9

Step 1

Select A1:

$1 \rightarrow 2$

Step 2

A2 starts at 3, which is after 2.

Select A2:

$3 \rightarrow 4$

Step 3

A3 starts at $0 \rightarrow$ overlaps, so reject.

Step 4

A4 starts at 5, after 4.

Select A4:

$5 \rightarrow 7$

Step 5

A5 starts at 8, after 7.

Select A5:

$8 \rightarrow 9$

Step 6

A6 starts at $5 \rightarrow$ overlaps with A4, so reject.

Greedy result

Selected activities:

$A1 \rightarrow A2 \rightarrow A4 \rightarrow A5$

Number of activities = 4

B. Dynamic Programming

DP calculates the maximum number of compatible activities by considering different combinations.

One optimal combination is:

A1 → A2 → A4 → A5

Number of activities:

4

Therefore, DP also produces:

Optimal solution = 4 activities

Comparison

Feature	Greedy	Dynamic Programming
Selected activities	A1, A2, A4, A5	A1, A2, A4, A5
Number selected	4	4
Time complexity	O(n log n)	Generally $O(n^2)$
Strategy	Earliest finish time	Evaluate subproblems
Complexity	Simple	More complex
Optimality	Guaranteed for standard activity selection	Guaranteed

Why is Greedy Correct?

Selecting the activity with the earliest finishing time leaves the maximum possible remaining time for other activities.

For example:

A1 (1–2) finishes earlier than A3 (0–6).

Choosing A1 leaves much more room for A2, A4, and A5.

Therefore, the earliest-finish-time rule satisfies the greedy-choice property for the standard activity-selection problem.

Overall Insights and Method Selection

Problem	Greedy	DP	Best Choice
0/1 Knapsack	Not always optimal	Optimal	DP
Fractional Knapsack	Optimal	Not necessary	Greedy
Shortest Path with non-negative edges	Optimal	—	Dijkstra
Shortest Path with negative edges	✕	Bellman-Ford	Bellman-Ford
MST	Prim	—	Prim/Kruskal
Coin Change [1,3,4]	Not optimal	Optimal	DP
Activity Selection	Optimal	Also optimal	Greedy

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____